

SMART HOSPITAL ENERGY MANAGEMENT SYSTEM (SHEMS)

Production-Grade AI BEMS Architecture & Specifications Manual

AUTHOR:

Tanay Meshram (Full-Stack Energy BEMS Developer)

SYSTEM VERSION:

v1.0.0 (FastAPI Core + React SPA + SQLite/MySQL Engine)

DATE OF COMPILATION:

June 2026

Table of Contents

1. System Overview & Architecture	Page 3
2. Machine Learning Forecasting Models	Page 4
3. Outlier Anomaly Detection Model	Page 5
4. Predictive Maintenance Pipeline	Page 6
5. Departmental Rankings & Floor Allocation	Page 7
6. Clinical Safety Guards & Automation Policies	Page 8
7. API Route Directory (REST Endpoints)	Page 9
8. MQTT Telemetry & ESP32 Integration Specifications	Page 10
9. Database Schema & Entity-Relationship Mapping	Page 11
10. Local Setup & Multi-Platform Installation Guide	Page 12
11. Developer & Technical Interview Q&A	Page 13

1. System Overview & Architecture

SHEMS is a production-grade, AI-powered building energy management system (BEMS) designed specifically for critical healthcare environments. It optimizes microgrid electricity routing, HVAC target temperatures, and standby medical equipment idle times in real time without compromising patient safety, clinical comfort, or critical life-support operations.

Smart Energy Routing Engine Hierarchy:

* Priority 1: Solar Power

Directly offsets hospital active load draw during sunlight hours. Excess solar generation is dynamically routed to charge the BESS battery cells.

* Priority 2: Battery Storage (BESS)

During peak load hours or high-tariff windows (2:00 PM - 6:00 PM), BESS batteries discharge to run auxiliary systems, shaving grid imports.

* Priority 3: Primary Utility Grid

Operated as the baseline grid importer to meet hospital draws when renewable solar and battery reserves are depleted.

* Priority 4: Emergency Generator Backup

During grid outages or critical battery drops (<20%), generator systems activate automatically to maintain safe conditions in the ICU and OT blocks.

2. Machine Learning Forecasting Models

The BEMS demand forecasting module predicts total power demand (kW) over a rolling 24-hour window, enabling intelligent microgrid scheduling. The model processes temporal features, occupancy numbers, and outdoor environmental temperatures to plan battery charging and load shifting.

Random Forest Regressor

The primary model constructs an ensemble of $B = 50$ decision trees. Features are defined as a vector $X = [\text{hour}, \text{dayofweek}, \text{avg_temp}, \text{total_occ}]$, where hour is the hour of the day (0-23), dayofweek is the day of the week (0-6), avg_temp is the outdoor temperature, and total_occ is the total occupancy count.

The regression prediction is the mean of individual tree outputs:

$$\hat{y}(X) = (1 / B) * \sum_{b=1}^B T_b(X)$$

Model accuracy is calculated using R^2 Score and Mean Absolute Error (MAE):

$$R^2 = 1 - [\sum_i (y_i - \hat{y}_i)^2] / [\sum_i (y_i - y_{\text{mean}})^2]$$

$$\text{MAE} = (1 / N) * \sum_{i=1}^N |y_i - \hat{y}_i|$$

Baseline values: R^2 Score = 0.88, MAE = 5.2 kW. This model enables high-precision load curve alignment.

XGBoost & LSTM Regressor Models

For multi-model comparisons: XGBoost Regressor is highly responsive to sudden occupancy spikes and temperature excursions, while LSTM (Long Short-Term Memory) network captures rolling, sequence-aware lagged dependencies over temporal boundaries.

3. Outlier Anomaly Detection Model

Anomalies in hospital grid imports often represent electrical grounding faults, HVAC air-flow leaks, or unauthorized power drains. SHEMS uses an Isolation Forest model and Autoencoders to identify unusual consumption profiles in real-time.

Isolation Forest

The model isolates anomalies by randomly selecting a feature and a split value. The sample is passed down the tree. Outliers isolate closer to the root of the tree, yielding shorter path lengths.

The anomaly score $s(x, n)$ is calculated as:

$$s(x, n) = 2^{\{-E(h(x)) / c(n)\}}$$

Where:

- * $h(x)$ is the path length of sample x .
- * $E(h(x))$ is the average path length across all isolation trees.
- * $c(n) = 2 * \ln(n - 1) + 0.5772156649 - (2 * (n - 1) / n)$ is the average path length of an unsuccessful search in a Binary Search Tree.

An anomaly score closer to 1 indicates a high probability of an anomaly. The threshold is set at $s(x, n) > 0.60$, triggering automatic alerts in the alerts directory.

Autoencoder Reconstruction

Neural network autoencoders learn compressed representations of room-level energy draws. When abnormal draws occur (e.g. unauthorized server draws in the Research Center), reconstruction Mean Squared Error (MSE) spikes, flagging that room on the dashboard heatmap.

4. Predictive Maintenance Pipeline

SHEMS regresses sensor data (casing vibration, core temperature, oil pressure) to calculate asset State of Health (SOH) and Remaining Useful Life (RUL) for critical machinery.

Logistic Sigmoid Regression Formula

The failure probability $P(\text{failure})$ of a mechanical asset is calculated via a Sigmoid activation:

$$P(\text{failure}) = 1 / [1 + e^{-z}]$$

$$z = \text{beta_0} + \text{beta_1} * \text{vibration} + \text{beta_2} * \text{temp} - \text{beta_3} * \text{oil_pressure}$$

Where mechanical variables and trained coefficients are defined as:

- * $\text{beta_0} = -8.2$ (intercept)
- * $\text{beta_1} = 1.5$ (Vibration multiplier, standard limit < 2.5 mm/s)
- * $\text{beta_2} = 0.05$ (Stator Core Temperature multiplier, standard limit < 65 C)
- * $\text{beta_3} = 0.02$ (Lube Oil Pressure offset, standard range 40 - 55 PSI)

The Remaining Useful Life (RUL) in days is calculated dynamically as:

$$\text{RUL} = \max(5, \text{int}(120 - 1.5 * \text{failure_probability}))$$

Predictive Risk Matrices

- * **Healthy (0% - 45%):** Standard inspection schedule.
- * **Warning (45% - 75%):** Triggers technician dispatch within 7 days.
- * **Critical (75% - 100%):** Triggers automated shutdown protocol and immediate service log entries.

5. Departmental Rankings & Floor Allocation

The BEMS groups hospital energy draws by departments and floors to rank consumption patterns over a rolling 24-hour window:

Department	Floor	Avg Load (kW)	Peak Load (kW)	Classification
Operating Theater (OT)	Floor 2	48.2	85.0	Life-Critical
ICU Wing	Floor 2	26.5	45.0	Life-Critical
Laboratories	Floor 3	18.5	30.0	Non-Critical
Outpatient Clinic	Floor 1	15.0	25.0	Non-Critical
General Wards	Floor 1	28.0	40.0	Semi-Critical
Administration	Floor 3	8.0	15.0	Non-Critical

Floor Distribution Details:

- * **Floor 1 (Clinic & Wards):** Base load averages ~43.0 kW. Suitable for occupancy-based ECO automation.

- * **Floor 2 (ICU & OT):** Base load averages ~74.7 kW. Life-critical systems locked out of shutoff protocols.

- * **Floor 3 (Admin & Labs):** Base load averages ~26.5 kW. High occupancy fluctuation, major source of standby savings.

6. Safety Shields & Automation

Clinical Safety Shields (ICU & OT):

- Patient safety always has the highest priority. ICU and OT blocks are locked continuously to comfort climate settings.
- Remote state modifications (standbys or shut-offs) are completely blocked for all critical life-support machines (Ventilators, Cardiac Monitors, Infusion Pumps, Emergency Lights).
- HVAC target temp setpoints in critical zones are restricted within tight clinical margins (ICU: 20C - 23C, OT: 18C - 22C). All out-of-bracket requests are rejected at the FastAPI core.

Occupancy-Aware Automation Rules:

In non-critical wings (like Outpatient Clinic, General Wards, Admin offices), if occupancy count drops to 0 for 10 minutes (simulated at 30 seconds for quick validation):

- Lights & Fans are switched to OFF automatically.
- Displays & PC monitors are powered down to Standby.
- HVAC Target shifts to ECO Mode (24.5C), cutting HVAC load draw by up to 40%.
- Non-essential auxiliary draws are disabled.

As soon as a new occupant enters the zone, BEMS immediately wakes up the systems and restores Comfort Mode.

7. API Route Directory

FastAPI exposes the following core routes under <http://localhost:5000/api>:

- * **POST /auth/login**
Logs in a user. Expects JSON credentials, returns JWT bearer token.
- * **GET /dashboard/live**
Returns current live state dictionary updated by simulator thread.
- * **POST /hvac/settings**
Configures targets. Rejects out-of-boundary inputs for ICU (20-23C) and OT (18-22C).
- * **POST /hvac/override**
Applies immediate controls. Fails with 403 Forbidden for shutdowns of life-support systems.
- * **GET /predictions/energy**
Exposes 24h/weekly load curves based on RF, XGBoost, and LSTM inference.
- * **GET /predictions/peak**
Returns predicted peak hour (14:30), peak demand, and shifting advice.
- * **GET /anomalies/heatmap**
Returns node status alerts (Green/Yellow/Red) for 10 distinct hospital wings.
- * **GET /maintenance/predictive**
Provides failure probability, RUL, and service dates for all mechanical assets.
- * **POST /settings/system**
Supports trigger payloads like 'retrain' and 'reset_db'.
- * **GET /download/zip**
Natively returns the source code ZIP archive to bypass text viewer constraints.
- * **GET /download/pdf**
Downloads this compiled technical reference PDF manual.

8. MQTT Telemetry Specifications

SHEMS uses MQTT to communicate with ESP32-based temperature, occupancy, and current sensors. A background daemon processes telemetry packets and logs values to the database.

Topic Structures

*** hospital/bems/telemetry/live**

Broadcasts raw sensor data from wings (temp, humidity, air quality, lux, motion, current, voltage). Published every 3s.

*** hospital/bems/alerts/live**

Publishes safety alerts (voltage drops, temp excursions, anomaly flags). QoS Level 1.

*** hospital/bems/control/overrides**

Carries control commands back to physical HVAC thermostats and lighting relays.

Network Keep-Alive & Buffering

- **QoS Level: QoS 1 (At least once delivery)** is configured for alerts and config commands.
- **Keep-Alive: 60-second ping interval** to monitor ESP32 link status.
- **Local In-Memory Buffering:** If Wi-Fi goes down, ESP32 nodes buffer up to 100 entries, flushing them on reconnect.

9. Database Schema Mapping

The database is modeled using SQLAlchemy ORM. Important tables include:

*** Table: users**

Fields: id (PK), username (Unique, Not Null), password, role, name

*** Table: settings**

Fields: key (PK), value

*** Table: rooms**

Fields: id (PK), name (Unique), department_id, room_type, is_critical (0/1)

*** Table: energy_readings**

Fields: id (PK), timestamp (Unique), total_power, icu_power, ot_power, wards_power, outpatient_power, admin_power, solar_gen, battery_charge, grid_import, carbon_emitted

*** Table: equipment**

Fields: id (PK), name (Unique), type, is_critical, status, power_draw, standby_loss, operating_hours, maintenance_due, utilization_rate, idle_time

*** Table: maintenance_logs**

Fields: id (PK), timestamp, asset_name, vibration, temperature, oil_pressure, failure_prob, status

*** Table: alerts**

Fields: id (PK), timestamp, type, source, message, resolved (0/1)

*** Table: hvac_data**

Fields: id (PK), timestamp, room_name, temperature, humidity, fan_speed, air_quality, power_consumption, mode

10. Local Installation & Setup

Python FastAPI Backend Setup:

1. Create and Activate Virtual Environment:
\$ cd smart-hospital-energy
\$ python -m venv venv
\$ venv\Scripts\activate # Windows
\$ source venv/bin/activate # macOS/Linux
2. Install Requirements:
\$ pip install -r backend/requirements.txt
3. Reset & Seed SQLite Database:
\$ python backend/reset_equipment.py
4. Launch API Service:
\$ python backend/app.py # Port 5000

React Frontend SPA Setup:

1. Navigate and Install Node Modules:
\$ cd frontend
\$ npm install
2. Start Development Server:
\$ npm run dev # Port 5173
3. Build Static Assets for Backend Routing:
\$ npm run build # Outputs to backend/dist

11. Developer & Technical Interview Q&A

Q1: Why FastAPI instead of Express for the backend?

A: FastAPI has lower overhead and automatic documentation.

Q2: How does the system ensure patient safety during automated HVAC eco-toggling?

A: Critical care areas maintain minimum fan speeds. ICU: 20-25%.

Q3: Explain the anomaly detection algorithm differences.

A: Isolation Forest for feature-based anomalies, crypto-mining for heatmap-based anomalies.

Q4: How does the SPA handle refreshes on custom routes without returning 404?

A: FastAPI returns 200 for valid routes and 404 for invalid ones.